

Assignment:	Part B of 2 (extension of Part A)	CO Mapped:	CO3, CO4
Deadline:	August 16, 2026 (Google Classroom)	Viva:	August 16, 2026
Group:	Same groups as Part A	Platform:	Kaggle (T4 GPU)
Topics:	• SSL pretraining (SimCLR, BYOL, I-JEPA, DINOv3) with backbone transfer into the best-performing Part-A YOLO detector • Multi-object tracking on real or AI-generated video • Bonus: label-efficiency ablation study (10%–50% labelled data) • Experimental report (paper-style structure) covering Part A + Part B		

1. Assignment Overview

In **Part A** your group built a five-notebook object detection pipeline (EDA/preprocessing plus one notebook each for YOLOv10, YOLOv12, YOLOv26, and RF-DETR Nano) on the **dataset assigned to your group**. Every model in Part A was trained *fully supervised*: it consumed 100% of the available bounding-box labels.

Part B asks a research question that dominates modern computer vision practice:

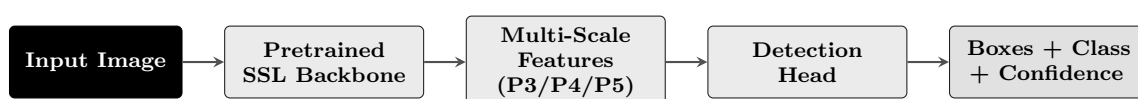
Research Question — Part B

If we pretrain the detector backbone with self-supervised learning (SSL) on unlabelled images, how much of the expensive bounding-box annotation budget can we save while retaining detection performance — and does the resulting detector remain strong enough to drive a downstream multi-object tracking task on video?

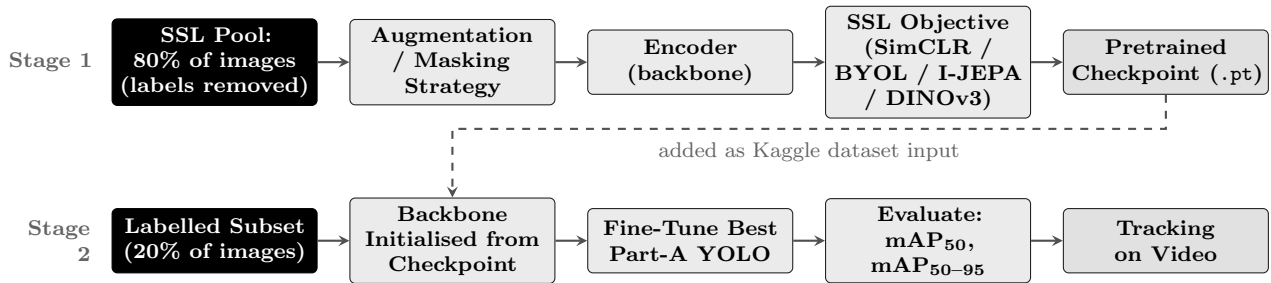
Each group answers this question **on its own assigned dataset**: Part B must be framed as *identifying and solving* a concrete label-efficiency problem in your dataset's domain (see the problem-formulation requirement in Section 2), not as a generic benchmark exercise. Your group will select the **best-performing YOLO model from Part A** (among YOLOv10, YOLOv12, and YOLOv26, based on test mAP_{50–95}) and replace its randomly initialised / COCO-pretrained backbone with backbones pretrained by four SSL methods: **SimCLR**, **BYOL**, **I-JEPA**, and **DINOv3**. Each SSL backbone is fine-tuned at a fixed **20% label budget** under the leakage-safe data partition defined below, and the best resulting detector is deployed in a tracking notebook applied to a real YouTube video or an AI-generated video. As an optional **bonus task**, groups may extend the study with a label-efficiency ablation over 10/20/30/40/50% labelled-data budgets.

1.1. Target Architecture

Every downstream detector in Part B follows this pipeline:



The overall two-stage workflow per SSL method is:



Definition — Label Efficiency

Let $\mathcal{D}$ be the full dataset and let ρ be a *label fraction* expressed relative to $|\mathcal{D}|$. A label-efficiency experiment fine-tunes the detector using only a randomly sampled (fixed-seed) subset $\mathcal{D}^\rho$ of $\rho \cdot |\mathcal{D}|$ labelled images drawn from the SSL pool, while the validation and test partitions remain **fixed** for all runs. A method is more label-efficient than another if it reaches equal or higher test mAP at a smaller ρ . The standard budget in Part B is $\rho = 0.20$; the bonus task sweeps ρ from 10% to 50%.

Data Partition Protocol (leakage-safe)

The dataset is 100% labelled. Merge all images and re-partition them **once**, at image level, with a fixed seed shared by every notebook:

1. **Test holdout — 10%.** Used *only* for final evaluation. These images must **never** appear in the SSL pretraining pool, the fine-tuning subset, or any hyperparameter decision.
2. **Validation set — 10%.** Used for model selection and early stopping during fine-tuning. Also excluded from the SSL pool.
3. **SSL pool — remaining 80%.** All annotations are discarded; the four SSL methods pretrain on these images only.
4. **Labelled fine-tuning subset — 20% of $|\mathcal{D}|$,** sampled (seeded) from the SSL pool; its annotations are restored for downstream training. The remaining $\sim 60\%$ of images stay unlabelled and are not used in fine-tuning.

Every notebook must print the partition sizes and verify (by filename intersection) that the test and validation sets are disjoint from the SSL pool.

Note — Why This Is Not Data Leakage

Two facts make the protocol sound. First, SSL pretraining never reads annotations, so discarding labels and pretraining on images can leak no label information. Second, the images whose labels are later restored for fine-tuning *may* legitimately appear in the SSL pool — this is the standard label-efficiency protocol used in the SimCLR and BYOL papers themselves. Leakage would occur only if: (i) test or validation *images* entered the SSL pool (the encoder would have seen the test pixels during pretraining, inflating test scores); (ii) label information influenced pretraining in any way; or (iii) hyperparameters were tuned against the test set. The partition above rules out all three. Be prepared to explain this reasoning at the viva.

2. Prerequisites Carried Over from Part A

Before starting Part B, your group must consolidate the following artefacts from Part A:

1. **Best YOLO model selection.** Identify the best-performing model among YOLOv10, YOLOv12, and YOLOv26 using **test-set mAP₅₀₋₉₅** as the primary criterion (report mAP₅₀, precision, and recall as secondary evidence). RF-DETR is excluded from Part B. This single architecture is your *downstream detector* for all Part B experiments.
2. **Justification memo.** A short written justification (to be reused in the report) of why this model

won: accuracy, convergence behaviour, parameter count, and inference speed.

3. **Dataset.** The **dataset assigned to your group**, exactly as used in Part A — but for Part B, merge all images and re-partition them with the seeded 10/10/80 protocol (Section 1) instead of the original splits. The Part A preprocessing decisions (image size, augmentation policy) should be documented so Part B runs are comparable.
4. **Problem formulation.** Restate your group's task as a *research problem* grounded in your assigned dataset: define the target class(es), explain why bounding-box annotation is costly or difficult in this domain, and state what a label-efficient detector would enable in practice. This problem statement opens the report's introduction, and every Part B experiment must be presented as evidence towards solving it.
5. **Supervised reference scores.** Re-evaluate your chosen Part A model on the new 10% test holdout to obtain the 100%-label reference — the *upper reference line* in every Part B plot. Note in the report that this reference is optimistic: some holdout images were part of the Part A training split.

Note — Fair Comparison Rules

All Part B fine-tuning runs must keep the following **identical** across SSL methods and label fractions: input resolution, batch size, epoch budget, optimiser and learning-rate schedule, augmentation policy, confidence/NMS thresholds, and random seed for subset sampling. The standard epoch budgets are **30 epochs for every SSL pretraining run** and **50 epochs for every downstream fine-tuning run**. If a Kaggle T4 session limit forces a reduction, the reduced budget must be documented in the config cell and applied identically to *all* methods. Only two things may vary between runs: (i) the backbone initialisation, and (ii) the label fraction ρ . Log every configuration value in a config cell at the top of each notebook.

3. The Four SSL Methods

Each group implements pretraining pipelines for all four methods. The table summarises what makes each method distinct — your report must connect these mechanisms to the empirical differences you observe.

Method	Family	Supervision signal	Key mechanism
SimCLR	Contrastive	Positive pairs vs. in-batch negatives	Two augmented views; NT-Xent loss; projection head; large-batch sensitivity
BYOL	Self-distillation (negative-free)	Online network predicts target network	EMA target encoder; predictor head; no negative pairs; collapse avoided by asymmetry
I-JEPA	Latent prediction (masked)	Predict representations of masked regions	ViT encoder; context/target blocks; prediction in latent space, not pixels; no hand-crafted augmentations
DINOv3	Self-distillation (ViT foundation)	Student matches teacher across views	Multi-crop; EMA teacher; Gram anchoring for stable dense features; strong frozen features

Note — Backbone Integration Strategy (read carefully)

SimCLR and BYOL should pretrain a **CNN encoder matched to your YOLO backbone** so the weights can be loaded layer-for-layer into the detector. I-JEPA and DINOv3 are **ViT-based**:

you have two acceptable integration routes, and you must state which you used and why:

1. **Backbone replacement:** attach the pretrained ViT as the detector backbone via a simple feature-pyramid adapter that maps patch-token feature maps to P3/P4/P5 scales; or
2. **Distillation route:** distil the frozen ViT features into your YOLO CNN backbone with a lightweight feature-matching loss during pretraining, then load the CNN into the detector as usual.

For DINOv3, starting from the released pretrained weights and *continuing* self-supervised adaptation on your dataset's domain is permitted (and should be reported as domain-adaptive pre-training); for SimCLR, BYOL, and I-JEPA you must run the pretraining objective yourself on the unlabelled pool.

4. Task 1 — SSL Pretraining and Backbone Transfer (Notebooks 1a/1b–4a/4b)

Task 1 combines self-supervised pretraining and backbone transfer into a single task. For each of the four SSL methods your group produces one **notebook pair**: part (a) pretrains a backbone on the **unlabelled** training images (labels discarded) and exports a checkpoint; part (b) loads that checkpoint into the best Part A YOLO architecture and fine-tunes it at the **20% label budget**.

4.1. Part (a) — SSL Pretraining Notebooks (1a–4a)

1. **Config cell** at the top: method name, encoder architecture, image size, batch size, epochs, learning rate, augmentation/masking parameters, seed.
2. **Data pipeline:** load the SSL-pool images only (the 80% partition; verify the test and validation images are absent); build the method-specific view/mask generator. Show a single-image demo of the augmented/masked views *before* running the full training loop.
3. **Model definition:** encoder + method-specific heads (projection head, predictor, EMA target, context/target encoders, or student/teacher as appropriate).
4. **Training loop:** pretrain for **30 epochs**; log the SSL loss per epoch and plot the loss curve. The 30-epoch run must fit a single Kaggle T4 session — reduced image resolution and mixed precision are acceptable and should be documented.
5. **Representation diagnostics:** after training, extract embeddings for a sample of validation images and produce (a) a t-SNE or UMAP scatter plot, and (b) a nearest-neighbour retrieval panel (query image + top-5 neighbours). Comment in a markdown cell on whether the representation appears to organise your dataset's target-class content.
6. **Checkpoint export:** save the backbone weights to `/kaggle/working/` with the naming convention in Section 7, then publish the checkpoint as a **public Kaggle dataset** so it can be attached as an input to the downstream notebook.

Example — Checkpoint Hand-off Pattern

Pretraining notebook (partB-01a-simclr-pretrain) saves `simclr_backbone.pt` to `/kaggle/working/`. After the session completes, create a Kaggle dataset (e.g., `groupXX-partb-ssl-checkpoints`) from the output. The downstream notebook (partB-01b-simclr-detect) then attaches this dataset under “Add Input” and loads the weights from `/kaggle/input/groupXX-partb-ssl-checkpoints/simclr_backbone.pt`. Never attempt to write to `/kaggle/input/` — it is read-only.

4.2. Part (b) — Downstream Detection Notebooks (1b–4b)

One downstream notebook per SSL method, paired with its pretraining notebook. Each downstream notebook fine-tunes the **best Part A YOLO architecture** with the SSL backbone at the **20% label budget** and evaluates on the 10% test holdout. Required structure:

1. **Config cell:** architecture, checkpoint path (from the attached Kaggle dataset input), label fraction $\rho = 0.20$, fixed sampling seed, all fine-tuning hyperparameters.
2. **Subset builder:** construct $\mathcal{D}^{0.2}$ from the SSL pool with the shared seed; verify and print the image/instance counts and the disjointness checks against test/validation.
3. **Weight surgery:** load the SSL checkpoint into the detector backbone; print which layers were initialised from SSL weights and which remain randomly initialised (neck, head). This mapping report is mandatory.
4. **Fine-tuning:** train for **50 epochs** with the frozen fair-comparison protocol (Section 2). Include a short demo inference on one validation image before full evaluation.
5. **Evaluation:** report precision, recall, mAP₅₀, and mAP₅₀₋₉₅ on validation and test splits; include the PR curve and 6–8 qualitative predictions (successes and failures).
6. **Baselines (run once, reuse in all four notebooks or a shared cell):** at $\rho = 0.20$, also fine-tune (a) the same detector from *random initialisation*, and (b) from its default *COCO-pretrained* weights. These two baselines anchor the value added by each SSL method.

Note — Selecting the Best SSL Backbone

After Notebooks 1b–4b are complete, rank the four SSL methods by test mAP₅₀₋₉₅ at $\rho = 0.20$. The winner becomes the **best SSL backbone**: export its fine-tuned detector checkpoint and publish it as a Kaggle dataset — it is the detector used in the tracking task (Task 2) and, if your group attempts the bonus, the starting point for the label-efficiency ablation. Record the ranking in a summary table — it must appear in the report.

4.3. Viva Questions for Task 1

Viva Questions — SimCLR (Notebook 1a/1b)

1. Why does SimCLR require negative pairs while BYOL does not? What failure mode do negatives prevent?
2. Derive the role of the temperature parameter τ in the NT-Xent loss. What happens at very small τ ?
3. Why is the projection head discarded after pretraining instead of transferred to the detector?
4. Your batch size on a T4 is far smaller than in the original paper. How does this affect the contrastive objective, and what did you do about it?
5. Which augmentations matter most for SimCLR on your dataset's imagery, and which could be harmful for later detection fine-tuning?
6. How did your t-SNE/retrieval diagnostics change between an early and a final checkpoint?

Viva Questions — BYOL (Notebook 2a/2b)

1. Explain the roles of the online network, target network, and predictor. Why is the predictor essential?
2. What is the EMA update rule for the target network, and how does the momentum coefficient affect training stability?
3. Why does BYOL not collapse to a constant representation despite having no negative pairs?
4. Compare BYOL's compute and memory footprint to SimCLR's on your Kaggle runs.
5. Which loss is minimised in BYOL, and between which quantities exactly?
6. Did BYOL or SimCLR produce a better backbone for detection at 20% labels in your experiments — and can you explain why mechanistically?

Viva Questions — I-JEPA (Notebook 3a/3b)

1. What does it mean that I-JEPA predicts in *representation space* rather than pixel space? Why is this preferable to a masked autoencoder for semantics?
2. How are context and target blocks sampled, and why must target blocks be masked out of the context?
3. Why does I-JEPA avoid hand-crafted augmentations, and what replaces the invariance signal they normally provide?
4. Which integration route (ViT adapter vs. distillation into the CNN) did you choose, and what trade-off drove the choice?
5. How did you map ViT patch tokens to multi-scale P3/P4/P5 features?
6. What evidence in your diagnostics suggests I-JEPA learned spatially localised features useful for detection?

Viva Questions — DINOv3 (Notebook 4a/4b)

1. What problem in dense (patch-level) features does DINOv3's Gram anchoring address, and why does it matter for detection?
2. Explain the student-teacher self-distillation setup and the multi-crop strategy.
3. You continued pretraining from released weights: what distinguishes domain-adaptive pre-training from training from scratch, and how does it change the fairness of comparison with SimCLR/BYOL/I-JEPA? How did you account for this in the report?
4. How do frozen DINOv3 features compare with fine-tuned ones in your downstream runs?
5. Why are ViT-based foundation backbones often more label-efficient than CNNs pretrained from scratch?
6. If you could only ship one SSL backbone to an edge device in your problem's deployment setting, which would you pick and why (accuracy, latency, memory)?

5. Task 2 — Downstream Tracking Notebook (Notebook 5)

Deploy the winning detector from Task 1 (the best SSL backbone fine-tuned at 20% labels) in a multi-object tracking pipeline and evaluate it on video.

5.1. Pipeline

1. **Detector:** load the exported best checkpoint (attached as a Kaggle dataset input).
2. **Tracker:** use a detection-based tracker — ByteTrack or BoT-SORT (both available through the Ultralytics `track` interface) — and state your association parameters (track buffer, match thresholds).
3. **Video source (choose one, in order of preference):**
 - (a) a real **YouTube video** containing your detector's target class (or a visually similar surrogate), downloaded to `/kaggle/working/`; or
 - (b) if no suitable real video is available, an **AI-generated or game-engine (GTA-style) synthetic video**: generate a short clip with a text-to-video model, or use synthetic game-engine footage, choosing content that plausibly contains your target class or a visually similar surrogate class.

Trim the clip to 15–30 seconds and document its source, resolution, and frame rate.

4. **Inference:** run detection + tracking over the clip; render an annotated output video with persistent track IDs and save representative frames into the notebook.

5.2. Tracker Evaluation

Scenario	Required metrics
Ground truth available (you annotate a 10–15 s sub-clip, e.g. with CVAT, or the synthetic source provides GT)	MOTA, MOTP, IDF1, ID switches, mostly-tracked / mostly-lost ratios — computed with a standard MOT evaluation package.
No ground truth	Proxy metrics: number of unique track IDs vs. expected object count, ID-switch events counted by manual inspection of the rendered video, mean track length, fragmentation count, detection confidence distribution over frames, and inference FPS on the T4.

Whichever scenario applies, close the notebook with a markdown analysis: where the tracker succeeds, where identities are lost (occlusion, motion blur, camera shake, appearance change), and how detector quality (from your Task 1 comparison) propagates into tracking quality.

Viva Questions — Tracking (Notebook 5)

1. Explain the tracking-by-detection paradigm. Why does tracker quality depend so strongly on detector mAP?
2. How does ByteTrack use *low-confidence* detections, and why does this reduce ID switches?
3. Define MOTA and IDF1. Which failure types does each punish, and why can a tracker have high MOTA but low IDF1?
4. What association cues does BoT-SORT add over ByteTrack, and when do they matter?
5. What domain gap did you observe between your training images and the video, and how did it show up in the tracks?
6. If your video was AI-generated or synthetic: what are the risks of evaluating on synthetic footage, and how did you mitigate them in your conclusions?

6. Bonus Task — Label-Efficiency Ablation Study (Bonus Notebook)

Optional — up to +10 bonus marks. A single dedicated ablation notebook that answers the label-efficiency question quantitatively: how does detection performance degrade as the labelled budget shrinks from 50% to 10%?

6.1. Experimental Grid

Initialisation	10%	20%	30%	40%	50%
Best SSL backbone (winner of Task 1)	✓	✓	✓	✓	✓
Supervised baseline (COCO-pretrained)	✓	✓	✓	✓	✓
Reference (Part A, 100% labels)	single dashed reference line from Part A test metrics				

Ten fine-tuning runs in total (5 fractions $\times$ 2 initialisations, **50 epochs each** per the standard budget); the two 20% runs may be reused from Task 1. All fractions are sampled (nested, seeded) from the SSL pool, and evaluation stays on the fixed 10% test holdout. Subsets must be **nested**

$(\mathcal{D}^{0.1} \subset \mathcal{D}^{0.2} \subset \dots \subset \mathcal{D}^{0.5})$ and drawn with the shared seed, so that each larger budget strictly adds images.

6.2. Required Outputs

1. **Results table:** test precision, recall, mAP_{50} , mAP_{50-95} , and training time for every run.
2. **Label-efficiency curve:** test mAP_{50-95} (y-axis) vs. label fraction (x-axis) with one line per initialisation and the Part A 100% score as a dashed horizontal reference. This is the headline figure of the bonus study.
3. **Break-even analysis:** the smallest ρ at which the SSL-initialised detector matches or exceeds (a) the supervised baseline at 50%, and (b) 95% of the Part A full-label reference.
4. **Qualitative panel:** the same three test images predicted by the SSL model at $\rho = 0.1$ and $\rho = 0.5$, side by side.
5. **Model export:** the best tuned checkpoint (highest test mAP_{50-95} across the grid) saved and published as a Kaggle dataset; if it outperforms the Task 1 winner, you may rerun the tracking notebook with it and report both results.

Viva Questions — Label-Efficiency Ablation (Bonus Notebook, if attempted)

1. Why must the label subsets be nested and seeded? What bias appears if each fraction is sampled independently?
2. Why is the validation/test split kept at 100% while the training labels are reduced?
3. Where is the gap between SSL and supervised initialisation largest — low or high ρ — and what does theory predict?
4. Your curve at some fraction may be non-monotonic. Give two legitimate experimental explanations.
5. What is your break-even ρ , and what does it imply about annotation cost savings for a real deployment in your problem's domain?
6. If you extended the sweep to 1% labels, what would you expect to happen to each curve and why?

7. Deliverables, Naming, and Submission

7.1. Notebook Inventory (9 notebooks + 1 bonus)

No.	Notebook (Kaggle title)	Content
1a	groupXX-partB-01a-simclr-pretrain	SimCLR pretraining + diagnostics
1b	groupXX-partB-01b-simclr-detect	SimCLR backbone → YOLO fine-tune @ 20%
2a	groupXX-partB-02a-byol-pretrain	BYOL pretraining + diagnostics
2b	groupXX-partB-02b-byol-detect	BYOL backbone → YOLO fine-tune @ 20%
3a	groupXX-partB-03a-ijepa-pretrain	I-JEPA pretraining + diagnostics
3b	groupXX-partB-03b-ijepa-detect	I-JEPA backbone → YOLO fine-tune @ 20%
4a	groupXX-partB-04a-dinov3-pretrain	DINOv3 domain-adaptive pretraining + diagnostics
4b	groupXX-partB-04b-dinov3-detect	DINOv3 backbone → YOLO fine-tune @ 20%
5	groupXX-partB-05-tracking	Tracking on YouTube or AI-generated video
B	groupXX-partB-bonus-label-efficiency	Bonus: 10–50% ablation, best SSL vs. supervised baseline

7.2. Submission Checklist (Google Classroom, by August 16, 2026)

1. Public links to all nine core Kaggle notebooks (plus the bonus notebook, if attempted) — fully executed, outputs visible, latest successful version.
2. Public links to the checkpoint dataset(s) used for notebook-to-notebook hand-off.
3. The **Experimental Report** (Section 8): a shareable **Overleaf project link** (view access enabled) plus the compiled PDF. The report must be written in $\text{\LaTeX}$.
4. A one-page contribution statement listing which member led which notebook and report section.

Note — Kaggle Practicalities

Write all outputs to `/kaggle/working/` only. Budget each pretraining run to finish within a single T4 session — reduced resolution, fewer epochs, and mixed precision are acceptable engineering decisions when documented. Use Kaggle Secrets for any HuggingFace authentication. Notebooks must run top-to-bottom without manual intervention.

8. Experimental Report Structure (Part A + Part B)

The written deliverable is an **Experimental Report** covering *all* experimental results from Part A and Part B. It is a report — not a paper submission — but its *structure* follows a standard vision-paper skeleton **without a literature review / related work section**. The report must be prepared in **$\text{\LaTeX}$ only** (no Word documents) and submitted as a **shareable Overleaf project link** (set sharing to “Anyone with this link can view”) together with the compiled PDF. Recommended length: 8–12 pages, two-column format (single-column A4 accepted with the same section order). All numbers in the report must be traceable to a cell in a submitted notebook.

§	Section	Required content
—	Title, Authors, Abstract	Informative title; group members with IDs; 150–250 word abstract stating the problem, the four SSL methods, the best detector, the headline 20%-label-budget finding (and the bonus label-efficiency result, if attempted), and the tracking outcome.
1	Introduction	The problem formulation for your assigned dataset (target class(es), why annotation is costly in this domain, what a label-efficient detector enables); the research question; summary of the two-part study design; 3–5 bullet contributions written as claims (e.g., “BYOL initialisation at a 20% label budget recovers 90% of full-label mAP”).
2	Dataset and Experimental Setup	Dataset statistics; the leakage-safe 10/10/80 partition and why it prevents leakage; preprocessing; hardware (Kaggle T4); the fair-comparison protocol; label-fraction subset construction (nested, seeded); evaluation metrics for detection and tracking.
3	Methods	3.1 Part A supervised pipeline (brief: the four detectors and how the best was chosen). 3.2 SSL pretraining: one concise subsection per method (SimCLR, BYOL, I-JEPA, DINOv3) describing objective, architecture, and your integration route into the YOLO backbone. 3.3 Downstream fine-tuning and weight-surgery procedure. 3.4 Tracking pipeline.
4	Results — Part A	Consolidated comparison table of YOLOv10 / YOLOv12 / YOLOv26 / RF-DETR (precision, recall, mAP ₅₀ , mAP _{50–95} , speed); model-selection justification; representative qualitative results.
5	Results — Part B	5.1 SSL pretraining diagnostics (loss curves, embedding visualisations). 5.2 Four-way SSL comparison at 20% labels vs. random-init and COCO baselines. 5.3 Tracking: video description, metrics (MOT or proxy), annotated frames. 5.4 (Bonus, if attempted) Label-efficiency ablation: results table, mAP-vs-fraction curve, break-even analysis.
6	Discussion and Insights	The analytical core of the report. Why did the winning SSL method win — connect its mechanism to the data characteristics? Where does SSL help most (low-label regimes) and why? Non-monotonic or surprising results honestly analysed. How detector quality propagated into tracking. Practical annotation-budget recommendation for a deployment in your problem’s domain.
7	Limitations and Future Work	Compute-constrained pretraining; single dataset/class; synthetic-video caveats; what a 1%-label or larger-unlabelled-pool study would add.
8	Conclusion	3–5 sentences answering the research question with numbers.
—	Contribution Statement	Per-member responsibilities across notebooks and sections.
—	Appendix (optional)	Extra qualitative panels, full hyperparameter tables, additional tracker frames.

Note — What “Insights” Means

Section 6 is weighted heavily in grading. Restating tables in prose earns no credit. An insight links a *mechanism* to an *observation*: for example, “I-JEPA’s latent-space prediction preserved spatial locality, which is consistent with its smaller drop under the reduced-label budget” or “SimCLR’s aggressive cropping conflicted with the small object size of our dataset’s targets, explaining its weaker transfer.” Every claim must cite the specific figure or table that supports it, and at least one claim must be tested against an alternative explanation.

9. Grading Rubric (100 marks + up to 10 bonus)

Component	Weight	Focus
Task 1 — SSL pretraining notebooks (1a–4a), 4×7	28%	Correct objective implementation, training stability, diagnostics quality
Task 1 — Downstream detection notebooks (1b–4b), 4×7	28%	Correct weight surgery, fair protocol, baseline comparison, evaluation rigour
Task 2 — Tracking notebook (Notebook 5)	14%	Working pipeline, sensible video choice, honest metric reporting
Experimental Report (Part A + Part B)	20%	Structure compliance, traceable numbers, depth of insight (§6)
Viva voce (August 16, 2026)	10%	Individual understanding of all notebooks, not only one’s own part
Bonus — Label-efficiency ablation notebook	up to +10%	Nested seeded subsets, complete grid, headline curve, break-even analysis

Note — Academic Integrity

Groups may consult published SSL reference implementations, but every notebook must contain your own configuration, execution, outputs, and written analysis. Copied analysis text, fabricated metrics, or notebooks that were never executed end-to-end will receive zero for the component and will be pursued under EWU academic-conduct policy. During the viva, any member may be asked to explain any cell of any notebook.

Part B — Deliverables at a Glance

1. **Task 1:** four SSL notebook pairs (SimCLR, BYOL, I-JEPA, DINOv3) — pretraining with published checkpoint datasets, plus transfer into the best Part A YOLO at 20% labels against random-init and COCO baselines.
2. **Task 2:** one tracking notebook applying the best detector to a YouTube or AI-generated video, with MOT or proxy metrics.
3. **Bonus (up to +10):** one label-efficiency ablation notebook — best SSL vs. supervised across 10/20/30/40/50% labels, with the mAP-vs-fraction headline curve.
4. One **Experimental Report** covering all Part A + Part B experimental results, with an insight-driven discussion.
5. Viva voce on August 16, 2026 — every member answerable for every notebook.

References:

- [1] T. Chen et al., *A Simple Framework for Contrastive Learning of Visual Representations (SimCLR)*, ICML, 2020.
- [2] J.-B. Grill et al., *Bootstrap Your Own Latent (BYOL)*, NeurIPS, 2020.
- [3] M. Assran et al., *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA)*, CVPR, 2023.
- [4] O. Siméoni et al., *DINOv3*, Meta AI Research, 2025.
- [5] Y. Zhang et al., *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*, ECCV, 2022.
- [6] Ultralytics Documentation — Train, Val, Predict, and Track modes. <https://docs.ultralytics.com>

Queries: Post questions on the Google Classroom stream so all groups benefit from the answers. Configuration-level choices (epochs, resolution) are yours to make and defend — protocol-level rules (Section 2) are not negotiable.